

Dokumentation

[High Level Team]

Inhaltsverzeichnis

Inhaltsverzeichnis.....	2
Einleitung.....	3
Installation.....	4
Modulübersicht.....	5
Allgemein.....	5
Simulation der Emotionen.....	5
Kommunikation.....	5
Gesichts- und Emotionserkennung via Webcam.....	6
Gesichtserkennung.....	6
Emotionserkennung.....	6
Umsetzung.....	7
Einrichtung der Umgebung.....	7
Implementierung der Gesichtserkennung.....	7
Implementierung der Emotionserkennung.....	7
ROS2.....	7
ROS-Pakete.....	8
Senden der Daten über einen Socket.....	8
Integration der Emotionserkennung via Webcam in das Projekt.....	9
Weitere nennenswerte Programme.....	11
Ideen für die Zukunft.....	12
Credits und grober Verlauf.....	13

Einleitung

Dieses Repository enthält die High-Level-Software für den Roboterkopf. Die Software erkennt Gesichter und die dargestellte Emotion, simuliert die Emotionen die der Roboter empfindet an Hand der aufgenommenen Audio, kommuniziert mit seinem Gegenüber basierend auf der simulierten Emotion und liefert Information über die aktuelle simulierte Emotion und die Position des erkannten Gesichts im Bild an das Low Level Team.

Dieses Dokument verschafft euch einen Überblick über das Repository und wie die einzelnen Module funktionieren. Des Weiteren wird auch erklärt, wie ihr das Repository bei euch lokal zum Laufen bringt, was die Anforderungen dafür sind und woran ihr als nächstes arbeiten könnt.

Installation

Der gesamte Installationsprozess ist in der ReadMe im Repository beschrieben. Bitte beachtet die Schritte ganz genau, dann dürfte auch alles problemlos funktionieren. Zuletzt war auf dem Rechner von Christoph im Makerspace die aktuelle Version des Projekts draufgeladen. Falls er die nicht runtergenommen hat, kann man auch da am Projekt arbeiten und es testen. Je nach Modellgröße ist es auch empfehlenswert, die Sachen im Makerspace zu testen, da die Modelle sehr viel Leistung beanspruchen. Der Rechner im Makerspace hatte zuletzt eine Nvidia rtx 4090, einen i9 der 14-ten Generation und 128 GB DDR5 Ram. Und selbst das ist nicht genug, wenn man die Größten 70B Modelle nutzen will.

Deswegen empfehlen wir für Llama:

- das 7B Modell im Makerspace
- 1B oder 3B am Heimrechner

Es ist auch noch zu beachten, dass ein Nvidia Grafikkarte vorausgesetzt wird für die High Level Software. Wenn ihr keine habt, dann wird das Programm nicht funktionieren.

Modulübersicht

Allgemein

Wir haben uns beim Erstellen des Codes in 4 Module aufgeteilt. Bilderkennung, Emotionen Simulation, Kommunikation und Schnittstelle zum Low Level Team via Ros2. Diese 4 Module kommunizieren miteinander via Events. Ansonsten sind die Module komplett unabhängig voneinander. Die Events setzen nur Werte in den Attributen der anderen Klassen. Wenn der Wert nicht geändert wird, dann bleibt er bei einem neutralen Wert. Im Folgenden gehen wir nochmals genauer auf die einzelnen Module ein. Alle Dateien im Projekt haben ebenfalls Kommentare, die nochmal die Abläufe der Klassen beschreiben.

Simulation der Emotionen

Für das Simulieren von Emotionen werden 3 verschiedene Dateien verwendet. *AudioRecorder_v3.py*, *WhisperSpeech2Text.py* und *Llama_emotions.py*.

Zuerst werden mit dem Audio-Recorder konstant Audiodaten aufgenommen. Der nimmt entweder alle 4 Sekunden auf oder bis die Person zu Ende gesprochen hat, je nachdem was früher eintritt.

Danach werden die Audiodaten an Whisper weitergegeben, um den Text aus der Sprache zu generieren. Die WhisperText2Speech Klasse hat ein Attribut mit erlaubten Sprachen, um Fehler zu vermeiden. Wenn die Sprache nicht übereinstimmt mit einer der erlaubten Sprachen, dann wird ein leerer Text ausgegeben. Dieser Text wird dann wiederum an den EmotionalAgent weitergeleitet.

Beim EmotionalAgent wird eine Llama Instanz erstellt, mit der Aufgabe, ein Dictionary mit den jeweiligen Emotionswerten zu erstellen. Danach machen wir nur ein bisschen Nachbearbeitung, damit immer das Format eingehalten wird. Falls wir einen leeren String erhalten, dann nähern wir uns iterativ der Default Emotion an. Der EmotionalAgent stellt 2 Events zur Verfügung, *on_emotions_changed* und *on_dominant_emotion_update*. Das erste Event liefert alle Emotionen in Form eines Arrays. Das zweite Event liefert nur den Namen der dominanten Emotion. Zum Erstellen der Emotion wird zusätzlich noch die erkannte Emotion von der Face Recognition verwendet. Falls wir das Event nicht subscriben, dann bleibt die FaceEmotion auf neutral.

Kommunikation

Bei der Kommunikation werden 2 verschiedene Dateien benötigt. *STTandTTS.py*, *WhisperSpeech2Text.py* und *AudioRecorder_v3.py*.

Der Audio-Recorder nimmt Audiodaten auf, ab dem Punkt, wo ein bestimmter Threshold überschritten wird. Das macht er so lange, bis der Threshold lange genug nicht mehr überschritten wird. Also in der Theorie immer dann, wenn eine Person spricht.

Danach werden die Audiodaten an Whisper weitergegeben, um den Text aus der Sprache zu generieren. Die WhisperText2Speech Klasse hat ein Attribut mit erlaubten Sprachen, um Fehler zu vermeiden. Wenn die Sprache nicht übereinstimmt mit einer der erlaubten Sprachen, dann wird ein leerer Text ausgegeben.

Der Text wird dann an Llama gegeben. Llama liefert uns dann eine passende Reaktion auf den Text. Als Sprache nutzt er entweder Englisch oder Deutsch, basierend auf der Sprache der Anfrage. Zusätzlich erhält Llama auch noch die Emotion von dem Emotional Agent, um in der Antwort die eigenen Emotionen mit darzustellen. Danach wird die Antwort in Audio umgewandelt und abgespielt. Während die Audio abgespielt wird, nimmt dieses Modul keine Audio auf, sondern wartet bis es zuende gesprochen hat. Danach fängt alles wieder von vorne an.

Gesichts- und Emotionserkennung via Webcam

Gesichtserkennung

Zur Gesichtserkennung wurden insbesondere die Bibliotheken OpenCV und dlib verglichen. OpenCV ist der De-facto-Standard für viele Anwendungen in der Bildverarbeitung und bietet zahlreiche Funktionen zur Gesichtserkennung. Dlib basiert teilweise auf OpenCV, bietet jedoch zusätzliche Features, die die Erkennung präziser machen. Aufgrund dieser Vorteile fiel die Wahl auf dlib für die Gesichtserkennung in diesem Projekt.

Emotionserkennung

Nachdem die Grundlagen für die Gesichtserkennung implementiert wurden, wurde eine geeignete Methode zur Emotionserkennung evaluiert. Hierfür wurden zwei Bibliotheken untersucht: TensorFlow und DeepFace.

- TensorFlow ist eine leistungsstarke und flexible Plattform für maschinelles Lernen. Sie ermöglicht sowohl das Training eigener Modelle als auch die Nutzung vortrainierter Modelle.
- DeepFace ist eine Open-Source-Bibliothek, die von der Community entwickelt wurde. Sie basiert auf TensorFlow und Keras und unterstützt verschiedene vortrainierte Modelle für die Emotionserkennung.

Obwohl TensorFlow aufgrund seiner Flexibilität eine erstklassige Wahl für solche Projekte ist, wurde aufgrund des Projektumfangs und der verfügbaren Zeit DeepFace als Lösung ausgewählt. DeepFace bietet eine schnelle und zuverlässige Emotionserkennung, ohne dass eigene Modelle trainiert werden müssen, was eine effiziente Implementierung ermöglicht.

Umsetzung

Einrichtung der Umgebung

Zunächst wurde eine virtuelle Umgebung eingerichtet, um alle Abhängigkeiten kompatibel und sicher zu verwalten. Für die Gesichtserkennung sind die Bibliotheken OpenCV (cv2) und dlib erforderlich. Da dlib spezielle Software Abhängigkeiten hat, mussten zusätzlich die Microsoft C++ Build Tools und CMake installiert werden. Nach der Installation dieser Programme konnten die benötigten Python-Pakete erfolgreich installiert werden.

Implementierung der Gesichtserkennung

Im Code wurden zuerst die benötigten Bibliotheken eingebunden. Anschließend wurde der Gesichtserkennungs-Detektor initialisiert und die Webcam gestartet. Der Videostream wird Frame für Frame ausgelesen, und für jeden Frame wird überprüft, ob ein Gesicht erkannt wird. Dazu wird das Bild in ein Graustufenbild umgewandelt, das anschließend von dlib analysiert wird. Der Detektor gibt eine Liste mit erkannten Gesichtern zurück.

Mit einer For-Schleife wird durch diese Liste iteriert, und die Koordinaten der erkannten Gesichter werden extrahiert. Um jedes erkannte Gesicht wird mithilfe der Koordinaten ein Rechteck gezeichnet. Schließlich wird das Bild mit den markierten Gesichtern ausgegeben. Dieser Prozess wird für jeden Frame wiederholt, sodass eine Echtzeit-Gesichtserkennung entsteht.

Implementierung der Emotionserkennung

Für die Emotionserkennung wurde die Bibliothek DeepFace eingebunden. Innerhalb der Schleife, die die Gesichter identifiziert, wurde eine Erweiterung vorgenommen: Es wird nur das nächstgelegene Gesicht (das mit dem größten Rechteck) für die Emotionserkennung verwendet. Dieser Gesichtsausschnitt wird an DeepFace übergeben, das den Frame mit einer umfangreichen Datenbank von Emotionsbildern abgleicht. Das Modell gibt die erkannten Emotionen mit einer prozentualen Gewichtung zurück.

Diese Emotionen können anschließend visuell auf dem Bild dargestellt werden, sodass die Emotionserkennung in Echtzeit angezeigt wird.

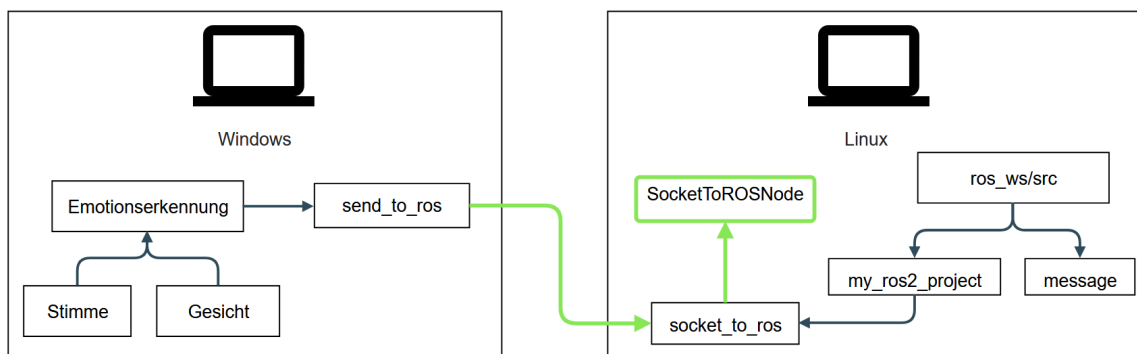
ROS2

Zur Kommunikation zwischen der Emotionserkennung und dem Low-Level-Team, das für die Steuerung der Roboter-Motoren zuständig ist, wird ROS2 verwendet. In diesem Abschnitt

wird die Vorgehensweise zur Erstellung der notwendigen ROS2-Nachrichten und zur Implementierung der Kommunikation beschrieben.

Die Kommunikation erfolgt über mehrere Systeme hinweg. Die Emotionserkennung und die Textausgabe laufen auf einem Windows-Gerät, das die erkannten Emotionen aus Stimme und Gesicht analysiert. Anschließend werden die verarbeiteten Daten per Socket an ein Linux-Gerät übertragen.

Auf dem Linux-Gerät wird ein Socket-Server gestartet (IP: 127.0.0.1, Port: 65432), der auf eingehende Daten wartet. Sobald das Windows-Gerät die analysierten Emotionen sendet, nimmt der Server die Daten entgegen und übergibt sie an ROS. Dort werden die empfangenen Werte veröffentlicht, sodass das Low-Level-Team sie für die Steuerung der Motoren weiterverarbeiten kann.



ROS-Pakete

Zunächst wurde ein ROS2-Paket erstellt, um die Nachricht zu definieren, die später vom Publisher-Node an den Empfänger-Node gesendet werden soll. Die Nachricht umfasst folgende Daten:

- Emotion (string): Die Emotion, die erkannt wurde.
- x (float): Die Distanz vom Gesicht zur Kamera.
- y (float): Die relative Verschiebung des Gesichts in x-Richtung (vom Mittelpunkt des Bildes).
- z (float): Die relative Verschiebung des Gesichts in y-Richtung.

Ein zweites Paket wurde erstellt, um die Daten der Emotionserkennung zu empfangen. Die Node „SocketToROSNode“ fungiert hier als Server und wartet auf eingehende Verbindungen über einen bestimmten Port.


```
def start_socket_server(self):
    HOST = '127.0.0.1' # IP-Adresse des Servers
    PORT = 65432       # Port für die Kommunikation
```

Die Daten werden als JSON-String empfangen und anschließend in das Nachrichtenformat des vorher definierten Pakets umgewandelt. Diese umgewandelte Nachricht kann anschließend veröffentlicht werden.

Um dies zu ermöglichen, müssen in der package.xml des ROS2-Pakets folgende Abhängigkeiten hinzugefügt werden:

```
<depend>message</depend>
<buildtool_depend>ament_python</buildtool_depend>
<exec_depend>rclpy</exec_depend>

<export>
| <build_type>ament_python</build_type>
</export>
```

Da der Node in Python geschrieben wurde, muss Ament-Python eingebunden werden, um das Paket korrekt zu bauen. Zudem muss das message-Paket eingebunden werden, damit der Nachrichtenaufbau bekannt ist.

Auch in der setup.py des Pakets müssen entsprechende Anpassungen vorgenommen werden, um sicherzustellen, dass das Python-Paket korrekt gebaut wird

```
install_requires=['setuptools'],
```

Senden der Daten über einen Socket

Um die Daten von VSCode an ROS2 zu senden, wurde der Ordner „Utils“ angelegt, der die Funktion „send_to_ros“ enthält.

```
def send_to_ros(emotion, distance_to_camera, relative_x, relative_y, debug=False):
    HOST = '127.0.0.1' # IP-Adresse des ROS2-Servers
    PORT = 65432       # Port für die Kommunikation
```

Diese Funktion empfängt die vier Daten (Emotion, x, y, z) und erstellt ein JSON-Datenobjekt. Nach der Erstellung des JSON-Objekts wird eine Verbindung zum Socket-Server aufgebaut, und die Daten werden über den angegebenen Port gesendet.

```
# Verbindung zum Server und Senden der Daten
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(data_string.encode('utf-8'))
    if debug:
        print(f'Sent to ROS: {data_string}')
```

Im Code zur Emotionserkennung wird der „Utils“-Ordner integriert, und es werden Parameter für die durchschnittliche Gesichtshöhe sowie die geschätzte Brennweite der Kamera in Pixeln angegeben.

```
KNOWN_FACE_HEIGHT_CM = 20.0 # Durchschnittliche Gesichtshöhe in cm
FOCAL_LENGTH = 300 # Geschätzte Brennweite der Kamera in Pixeln
```

Diese Werte sind entscheidend, um die ungefähre Distanz vom Gesicht zur Kamera zu berechnen.

Mit den genannten Parametern wird die Distanz des Gesichts zur Kamera sowie die relative Verschiebung in x- und y-Richtung berechnet. Die Bildmitte dient dabei als Referenz, von der aus die relativen Abstände in beide Richtungen bestimmt werden.

```
# Bildmaße
img_height, img_width = frame.shape[:2]
img_center_x = img_width // 2
img_center_y = img_height // 2
```

Der Abstand des Gesichts, das sich am nächsten zur Kamera befindet, sowie die relative Verschiebung vom Mittelpunkt des Bildes werden ermittelt.

```
# Berechnung der unteren Mitte des Rechtecks
bottom_middle_x = x + w // 2
bottom_middle_y = y + h

# Entfernung zur Kamera berechnen
distance_to_camera = (KNOWN_FACE_HEIGHT_CM * FOCAL_LENGTH) / h

# Relative Position der unteren Mitte zur Bildmitte
relative_x = bottom_middle_x - img_center_x
relative_y = img_center_y - bottom_middle_y # Y-Koordinatensystem umkehren
```

Nach erfolgreicher Emotionserkennung und Berechnung der Position des Gesichts wird die „send_to_ros“-Funktion aufgerufen. Diese überträgt die berechneten Daten (Emotion, x, y, z) an ROS2, wo sie dann vom Low-Level-Team weiterverarbeitet werden können.

```
# Daten an ROS senden
send_to_ros(emotion, distance_to_camera, relative_x, relative_y)
```

Integration der Emotionserkennung via Webcam in das Projekt

Um die oben beschriebenen Schritte in das Projekt zu integrieren, wurde die Gesichts- und Emotionserkennung als eigene Klasse implementiert. Diese Klasse enthält die Methoden `init`, `start_recognition` und `stop_recognition` sowie die Variablen `x`, `y` und `z`, die zur Übertragung der Positionsdaten an ROS2 dienen.

Die Funktion `send_to_ros` wurde in die Hauptfunktion des Projekts verlagert, sodass sie die zusammengefasste Emotion aus Sprach- und Gesichtserkennung an ROS2 weiterleitet.

Da die Emotionserkennung parallel zur Spracherkennung und Sprachausgabe laufen soll, wird sie in einem separaten Thread ausgeführt. Dies stellt sicher, dass das Hauptprogramm nicht blockiert wird und reaktionsfähig bleibt. Um die Rechenlast zu reduzieren, wurde die Bildrate auf 5 Bilder pro Sekunde begrenzt, ohne die Erkennungsgenauigkeit oder Nutzererfahrung zu beeinträchtigen.

Da die Emotionserkennung gleichzeitig mit der Spracherkennung und der Sprachausgabe laufen soll, wird sie in einem separaten Thread ausgeführt. Dies sorgt dafür, dass das Hauptprogramm nicht blockiert wird und weiterhin flüssig arbeitet. Um eine unnötig hohe Rechenlast zu vermeiden, wurde die Bildrate auf 5 Bilder pro Sekunde begrenzt.

Weitere nennenswerte Programme

Neben den oben genannten Modulen haben wir noch weitere Programme erstellt, welche bei der Arbeit am Projekt hilfreich sein könnten. Des Weiteren gibt es auch Programme, die noch nicht zufriedenstellend funktionieren, aber eine Verbesserung des Status Quo hervorbringen könnten. Folgende Dateien sind hier konkret zu nennen. *AudioMerger.py*, *BarChart.py*, *NoiseCancelling.py* und *TimeMeasurement.py*.

Audio Merger kann verwendet werden, um ganz einfach zwei .wav Dateien übereinander zu legen. Es wurde von uns verwendet um Noise mit Sprache zu mergen um das Noise Cancelling zu testen. Einfach `input_audio1`, `input_audio2` und `output_audio` nach belieben ändern und auf play drücken.

BarChart ist eine Klasse zum Darstellen von, wär hätte es gedacht, einen Bar Chart und ihn von Runtime zu verändern. Er wurde zu Beginn verwendet, um die Emotionen des Roboters zu visualisieren. Leider funktionierte es ab irgendeinem Zeitpunkt nicht mehr. Wir konnten es auch leider nicht beheben. Hier könnte man ansetzen und es wieder zum Laufen bringen. In dem Demo Video zum emotionalen Empfinden kann man sehen, wie es funktioniert hat.

Noise Cancelling verwendet ein Modell, um ungewollte Geräusche aus Audiodateien zu entfernen. Es hat schon ziemlich gut funktioniert, jedoch war es für Whisper eher störend als hilfreich. Wenn man es jedoch noch ein bisschen verbessern könnte, würde es vielleicht die Texterkennung nochmal verbessern. Eine Idee wäre auch, es vielleicht zu nutzen, um besser zu erkennen, wann eine Person gerade spricht. Momentan ist nämlich beim Audio-Recorder noch das Problem, dass er bei Räumen mit starken Lautstärkeschwankungen entweder immer aufnimmt oder gar nicht.

Time Measurement ist ein weiteres nützliches Werkzeug. Man kann mit den Funktionen `tic` und `tac` einen Timer starten und stoppen, um die Performance zu ermitteln. Es wurde von uns genutzt, um Response Time zu messen und die Bottlenecks zu ermitteln.

Ideen für die Zukunft

Das Projekt hat bereits viele Features. Es gibt jedoch ein paar Sachen, die man verbessern und implementieren könnte.

Die Antworten des Roboters klingen manchmal sehr erzwungen oder erfunden. Da könnte man an dem Prompt noch feilen. Der Kommunikation Agent hat außerdem die Möglichkeit, vordefinierte Antworten zu liefern. Dort könnte man weitere Funktionen hinzufügen, um das Gespräch ein bisschen mehr zu leiten.

Des Weiteren könnte man dem Agent die Möglichkeit geben einen Emotionsverlauf für die Mimik erstellen zu lassen. Z.B. [1s] Happy -> [0.2s] Surprised etc. Momentan wird immer nur die dominante Emotion genutzt, die ständig neu berechnet wird, während man mit dem Kopf redet.

Man könnte ihm auch noch die Emotion des Gegenübers mitgeben oder vielleicht sogar eine Beschreibung der letzten 10 Sekunden, die er gesehen hat. Da müsste man aber gucken, dass die Response Time nicht zu hoch wird, weil die jetzt schon eigentlich am Limit ist.

Unter *Modulübersicht->Weitere nennenswerte Programme* wurden ebenfalls einige Punkte aufgelistet, wo man ansetzen könnte.

Eine weitere Idee wäre, aus dem Kopf einen echten Charakter zu formen. Ihm also einen Namen und eine Persönlichkeit geben. Dabei wäre dann vielleicht auch wichtig für alle unterstützten Sprachen eine ähnliche Stimme zu wählen. Aktuell hat der Roboter im Deutschen eine männliche und im Englischen eine weibliche Stimme.

Was man auch noch verbessern könnte, wäre der Treshold im Audio-Recorder. Dieser wird nur beim Start initialisiert durch die Ermittlung des Mittelwerts der ersten X Sekunden. Die Lautstärke verändert sich jedoch über die Zeit, je nachdem wie viele Personen im Raum sind und wie Laut sie reden. Es sollte also regelmäßig neu ermittelt werden, oder man könnte ein Modell darauf trainieren, zu erkennen, ob jemand gerade redet. Alternativ könnte man es vielleicht auch durch die Gesichtserkennung bestimmen, ob jemand gerade redet. Diese Verbesserung ist neben den anderen wahrscheinlich die wichtigste und sollte garantiert umgesetzt werden.

Credits und grober Verlauf

Die vier Module wurden zu Beginn auf die vier Mitglieder des High Level Teams verteilt.

- Simulation der Emotionen: Jakub Kobedza
- Gesichtserkennung: Frederic Schwarz
- Kommunikationssystem: Simon Bilecen
- Ros2 Schnittstelle: Shady Madian

Nach Weihnachten übernahm Frederic Schwarz die Aufgaben von Shady Madian. Der Hauptgrund war, dass Shady uns mehrmals mitgeteilt hat, dass er Probleme mit den Aufgaben hat, aber gleichzeitig nicht zu Meetings erschien und die Hilfe Vorschläge nicht wahrgenommen hat. Zusätzlich wurde die Zeit langsam knapp, weshalb die Verantwortung verlagert wurde.

Gegen Ende hat Jakub Kobedza dann noch die Performance verbessert und mit Frederic Schwarz die Software auf dem Rechner im Makerspace aufgesetzt. Die Dokumentation und ReadMe wurde ebenfalls von Jakub Kobedza und Frederic Schwarz erstellt. Am Ende haben folgenden Personen an folgenden Dateien gearbeitet:

- alles Ros2 relevante: von Shady Madian erstellt, von Frederic Schwarz fertiggestellt und überarbeitet
- AudioMerger.py, AudioRecorder_v3.py, BarChart.py, Llama_emotions.py, NoiseCancelling.py, TimeMeasurement.py, WhisperSpeech2Text.py, setup.bat, setup.sh: von Jakub Kobedza erstellt
- STTandTTS.py: von Simon Bilecen erstellt, von Jakub Kobedza angepasst (Performance Verbesserungen)
- face_recognition_webcam.py: von Frederic Schwarz erstellt
- README.md, Dokumentation IP High Level.pdf, main.py: von Jakub Kobedza und Frederic Schwarz erstellt

Einige Dateien wurden aussortiert, da sie keine Relevanz mehr hatten. Dabei ist aber vielleicht nennenswert, dass vor allem die STTandTTS.py einige Iterationen durchlebt hat, an denen Simon Bilecen alleine gearbeitet hat, bevor wir uns auf die finale Version geeinigt haben.

In den Commits kann man auch sehen, wie oft wer etwas committed hat. Hier sind die Namen von GitHub, damit sich jeder selbst ein Bild davon machen kann.

- KeinPlanDiggi: Jakub Kobedza
- fresch09009: Frederic Schwarz
- SimonBilecen: Simon Bilecen
- shadymadian: Shady Madian